1 A Data Odyssey: Relational DBMS vs. GGG



Data Rep. in RDBMS

Real World Entity

Conceptual Level | Entity-Relationship Model

| Logical Level | Relational Model

| Physical Level | SQL

Computable Entity



Data Rep. in GGG

Real World Entity

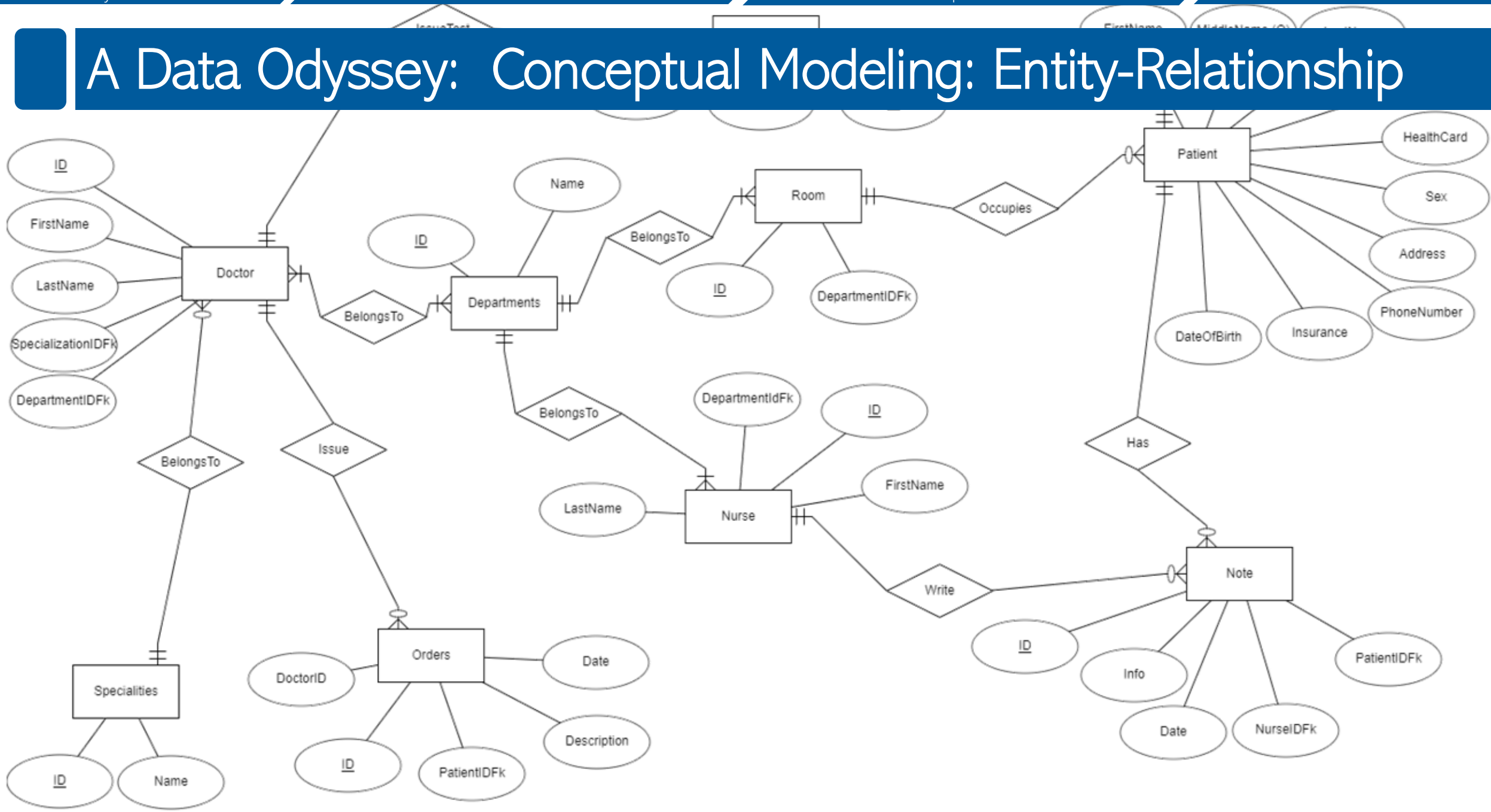
Conceptual Level | Web Ontology Language (OWL)

| Logical Level | RDF & Description Language

| Physical Level | SPARQL

| Computable Entity

A Data Odyssey: Conceptual Modeling: Entity-Relationship



3

A Data Odyssey: Relational DBMS vs. GGG



Data Rep. in RDBMS

Real World Entity

Conceptual Level | Entity-Relationship Model

| Logical Level | Relational Model

| Physical Level | SQL

Computable Entity



Data Rep. in GGG

Real World Entity

Conceptual Level | Web Ontology Language (OWL)

| Logical Level | RDF & Description Language

| Physical Level | SPARQL

| Computable Entity

A Data Odyssey: Logical Modeling: Relational

R1: Patient (Id, Number, FirstName, MiddleName, LastName, Sex, DateOfBirth, Address, Phone, HealthCardNo, InsuranceName), CK={Number} & Mandatory
R2: Doctor (Id, (MINC, FirstName, LastName, SpecialtyId), CK={MINC} & Mandatory, FK={SpecialtyId} & Mandatory
R3: Room (Id, Number, DepartmentId), CK={Number} & Mandatory, FK={DepartmentId} & Mandatory
R4: Nurse (Id, FirstName, LastName, DepartmentId), FK={DepartmentId} & Mandatory
R5: Test (Id, Name), CK={Name} & Mandatory
R6: Order (Id, Description, Date, DoctorId), FK={DoctorId} & Mandatory
R7: Specialty (Id, Name), CK={Name} & Mandatory
R8: Department (Id, Name), CK={Name} & Mandatory
R9: PatientRoom (Id, PatientId, RoomId, StartDate, EndDate), FK1={PatientId}, FK2={RoomId}, CK={PatientId, RoomId, StartDate}
R10: PatientNurse (Id, PatientId, NurseId, Date, Info), FK1={PatientId}, FK2={NurseId}, CK={PatientId, NurseId, Date}
R11: DoTest (Id, DoctorId, PatientId, TestId, Result, Date), FK1={PatientId} & Mandatory, FK2={DoctorId} & Mandatory, FK3={TestId} & Mandatory, CK={DoctorId, PatientId, TestId, Date}
R12: DoctorDepartment (Id, DoctorId, DepartmentId), FK1={DoctorId}, FK2={DepartmentId}, CK={DoctorId, DepartmentId}

5

A Data Odyssey: Relational DBMS vs. GGG



Data Rep. in RDBMS

Real World Entity

Conceptual Level | Entity-Relationship Model

| Logical Level | Relational Model

| Physical Level | SQL

| Computable Entity



Data Rep. in GGG

Real World Entity

Conceptual Level | Web Ontology Language (OWL)

| Logical Level | RDF & Description Language

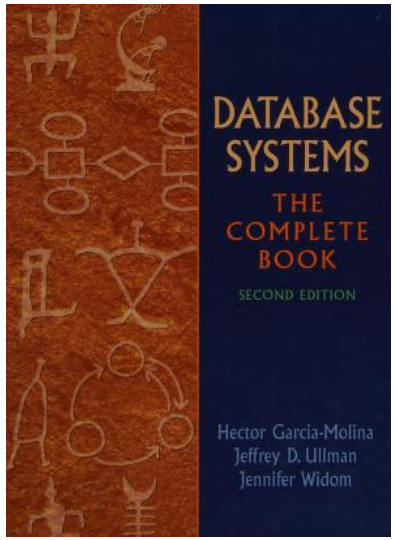
| Physical Level | SPARQL

| Computable Entity



The Matrix (1999), He is the One.

Outline: W07: Physical Modeling: SQL



CH06 (2nd Ed.)

I) Introduction

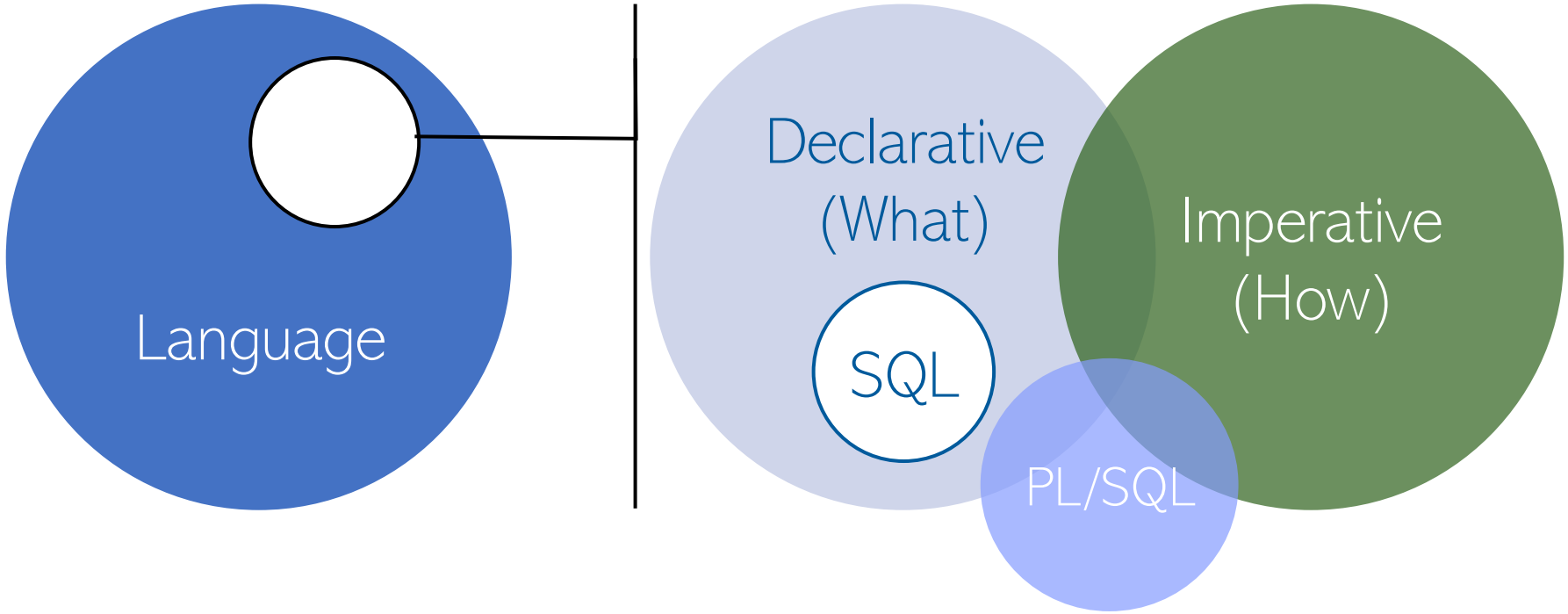
II) Data Definition Language (DDL)

III) Data Modification Language (DML)

IIII) Data Query Language (`SELECT`)

Hands-On

Programming Languages: SQL is Declarative

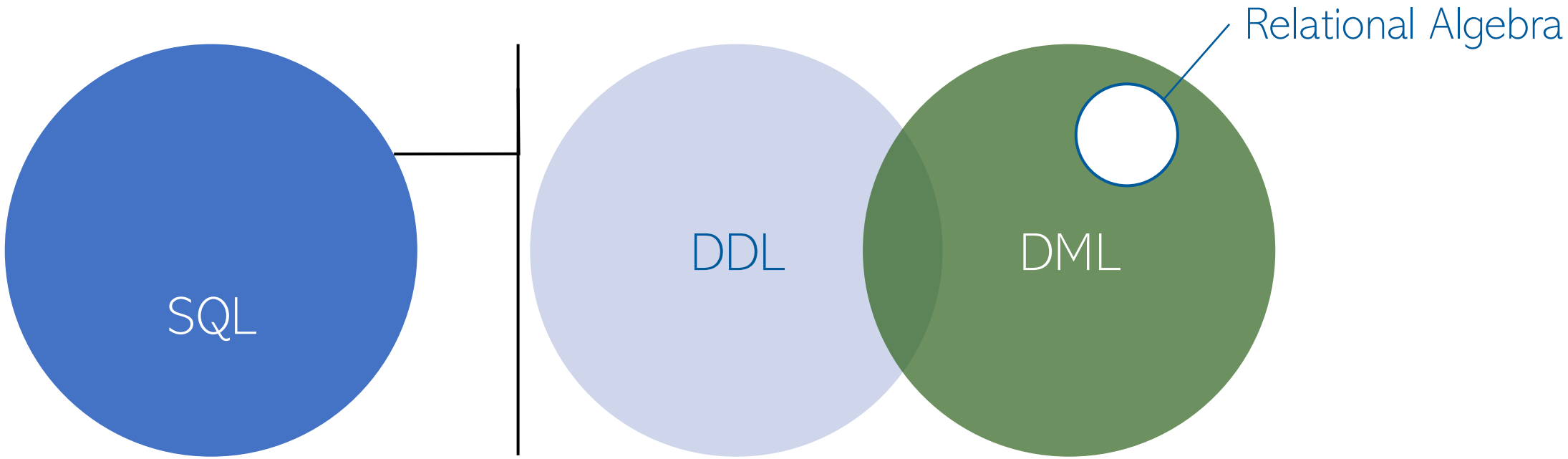


Programming Languages: SQL is Standardized

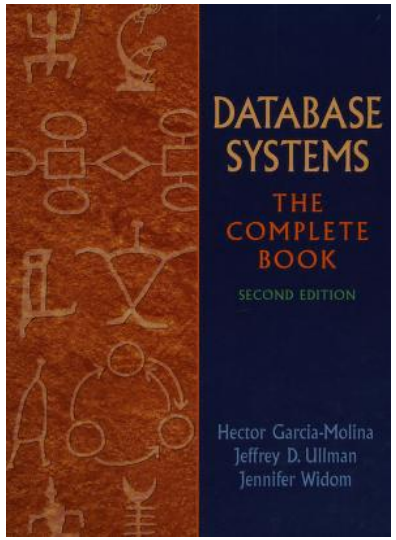
The most commonly used relational DBMS's query and modify the database through a language called SQL (sometimes pronounced "sequel"). SQL stands for "Structured Query Language." The portion of SQL that supports queries has capabilities very close to that of relational algebra. However: SQL also includes statements for modifying the database (e.g., inserting and deleting tuples from relations) and for declaring a database schema. Thus, SQL serves as both a data manipulation language (DML) and as a Data Definition Language. SQL also standardizes many other database command. There are many different dialects of SQL. First, there are three major standards. There is ANSI (American National Standards Institute) SQL and an updated standard adopted in 1992, called SQL-92 or SQL2. The recent SQL-99 (previously referred to as SQL3) standard extends SQL2 with object-relational features and a number of other new capabilities. Then, there are versions of SQL produced by the principal DBMS vendors. These all include the capabilities of the original .ITS1 standard. They also conform to a large extent to the more recent SQL2, although each has its variations and extensions beyond SQL2, including some of the features in the SQL-99 standard. Herein, we consider SQL as a stand-alone query language. SQLite does not fully support SQL-92.

10

SQL: DDL + DML



Outline: W07: Physical Modeling: SQL



CH06 (2nd Ed.)

I) Introduction

II) Data Definition Language (DDL)

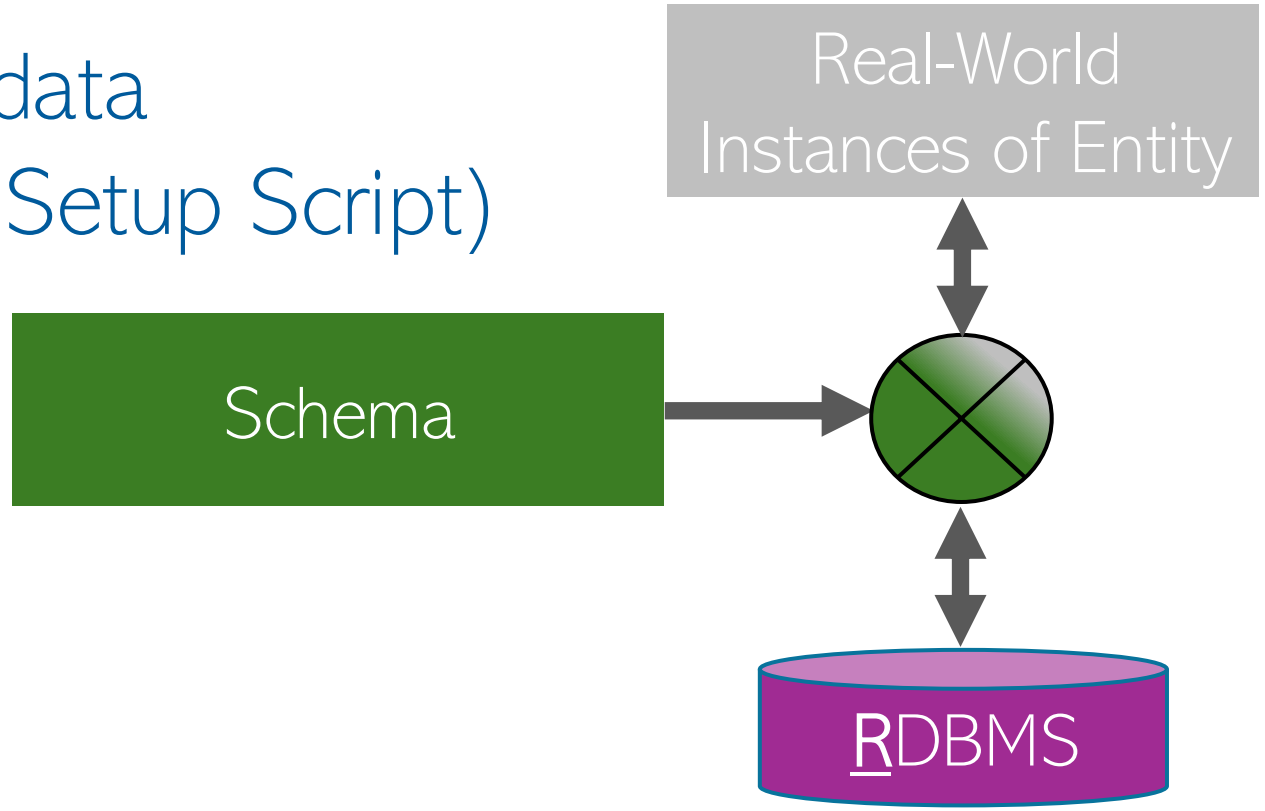
III) Data Modification Language (DML)

IIII) Data Query Language (`SELECT`)

Hands-On

SQL: DDL for Database, Tables, Constraints

Data Schema
Data about Data: Meta-data
Defined at Setup Time (Setup Script)
Rarely Change



SQL: DDL for Database, Tables, Constraints

```
CREATE DATABASE DatabaseName;
```

In SQLite > sqlite3 DatabaseName;

```
DROP DATABASE DatabaseName;
```

In SQLite > simply delete database file

```
ALTER DATABASE DatabaseName ... ;
```

In SQLite > no command!

```
RENAME DATABASE DatabaseName TO NewDatebaseName;
```

In SQLite > simply rename database file!

SQL: DDL for Database, Tables, Constraints

```
CREATE TABLE TableName(
        ColumnName1 DataType [NULL | NOT NULL],
        ColumnName2 DataType [NULL | NOT NULL],
        ...,
        ColumnNameN DataType [NULL | NOT NULL],
);
```

SQL: DDL for Database, Tables, Constraints

`INTEGER`

`DECIMAL(i, j)` i total number of digits, j number of digits after decimal point, e.g., 12.351 ∈ `DECIMAL(5,3)`

`FLOAT | REAL` single precision real number

`DOUBLE` double precision real number

`DATE` year month day, the format can be modified, e.g., yy-mm-dd | mm-dd-yyyy

`TIME` hour:minute:second, the format can be modified, precision depends on DBMS

`DATETIME`

`TIMESTAMP`

`CHAR(n)` n character, if less, padded with space

`VARCHAR(n)` n character max, if less, no padding

`TEXT` document, article, …

`BIT` only one bit, 0 | 1

`BOOLEAN` `TRUE | FALSE`

`BLOB` large object in binary format (voice, movie, image, …)

Data types are highly dependent on the DBMS.
See manual of the DBMS. e.g., https://www.sqlite.org/datatype3.html

16 SQL: DDL for Database, Tables, Constraints: Surrogate Key

```
CREATE TABLE TableName(
        Id INTEGER PRIMARY KEY AUTOINCREMENT
        ColumnName2 DataType [NULL | NOT NULL],
        ...,
        ColumnNameN DataType [NULL | NOT NULL],
);
```

17 SQL: DDL for Database, Tables, Constraints: Candidate Key

```
CREATE TABLE TableName(
        Id INTEGER PRIMARY KEY AUTOINCREMENT
        ColumnName2 DataType [NULL | NOT NULL],
        ...,
        ColumnNameN DataType [NULL | NOT NULL],
        CONSTRAINT UK_ColumnNames UNIQUE(candidate key columns)
);
```

SQL: DDL for Database, Tables, Constraints: Foreign Key

```
CREATE TABLE TableName(
        Id INTEGER PRIMARY KEY AUTOINCREMENT
        ColumnName2 DataType [NULL | NOT NULL],
        ...,
        ColumnNameN DataType [NULL | NOT NULL],
        CONSTRAINT UK_ColumnNames UNIQUE(candidate key columns)
        CONSTRAINT FK_ColumnNames FOREIGN KEY(foreign key columns)
        REFERENCES TargetTable(TargetTable's primary key columns)
);
```

SQL: DDL for Database, Tables, Constraints: Foreign Key

```
CREATE TABLE Movie(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        Title VARCHAR(255),
        ReleaseDate DATE,
        Language VARCHAR(255),
        RunningTime INTEGER,
        DirectorId INTEGER NOT NULL,
        CONSTRAINT UK_Title UNIQUE(Title),
        CONSTRAINT FK_Movie_DirectorId_2_Director_Id FOREIGN KEY(DirectorId)
        REFERENCES Director(Id)
);
CREATE TABLE Director(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        FirstName VARCHAR(255) NOT NULL,
        LastName VARCHAR(255) NOT NULL,
        CONSTRAINT UK_FirstName_LastName UNIQUE(FirstName, LastName)
);
```

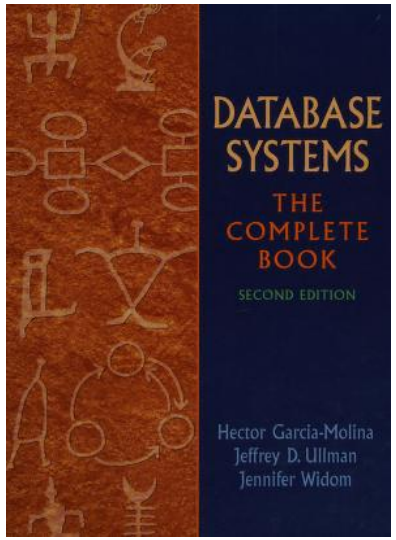


SQL: DDL for Database, Tables, Constraints: Foreign Key

```
CREATE TABLE Director(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        FirstName VARCHAR(255) NOT NULL,
        LastName VARCHAR(255) NOT NULL,
        CONSTRAINT UK_FirstName_LastName UNIQUE(FirstName, LastName)
);
CREATE TABLE Movie(
        Id INTEGER PRIMARY KEY AUTOINCREMENT,
        Title VARCHAR(255),
        ReleaseDate DATE,
        Language VARCHAR(255),
        RunningTime INTEGER,
        DirectorId INTEGER NOT NULL,
        CONSTRAINT UK_Title UNIQUE(Title),
        CONSTRAINT FK_Movie_DirectorId_2_Director_Id FOREIGN KEY(DirectorId)
        REFERENCES Director(Id)
);
```



Outline: W07: Physical Modeling: SQL



CH06 (2nd Ed.)

I) Introduction

II) Data Definition Language (DDL)

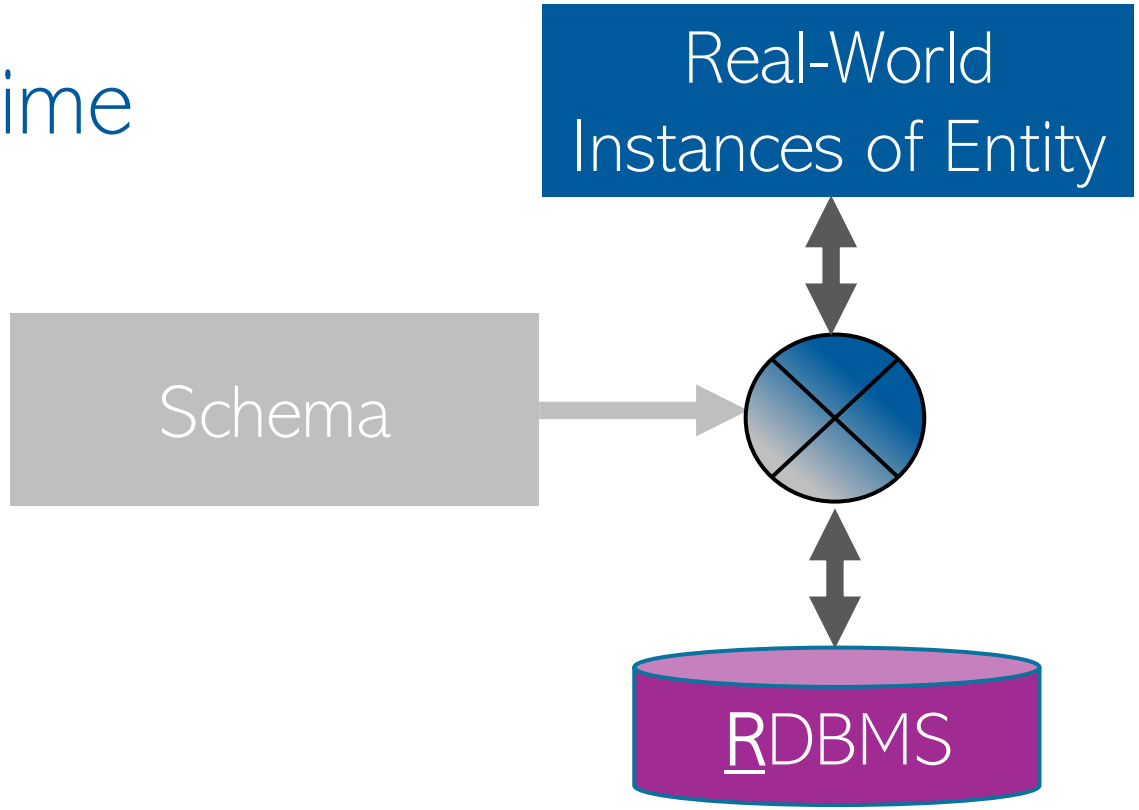
III) Data Modification Language (DML)

IIII) Data Query Language (`SELECT`)

Hands-On

SQL: DML for Entity Instances (Tuples)

Actual Entities (Tuple)
Rapidly Change at Running Time
Must Conform to Schema

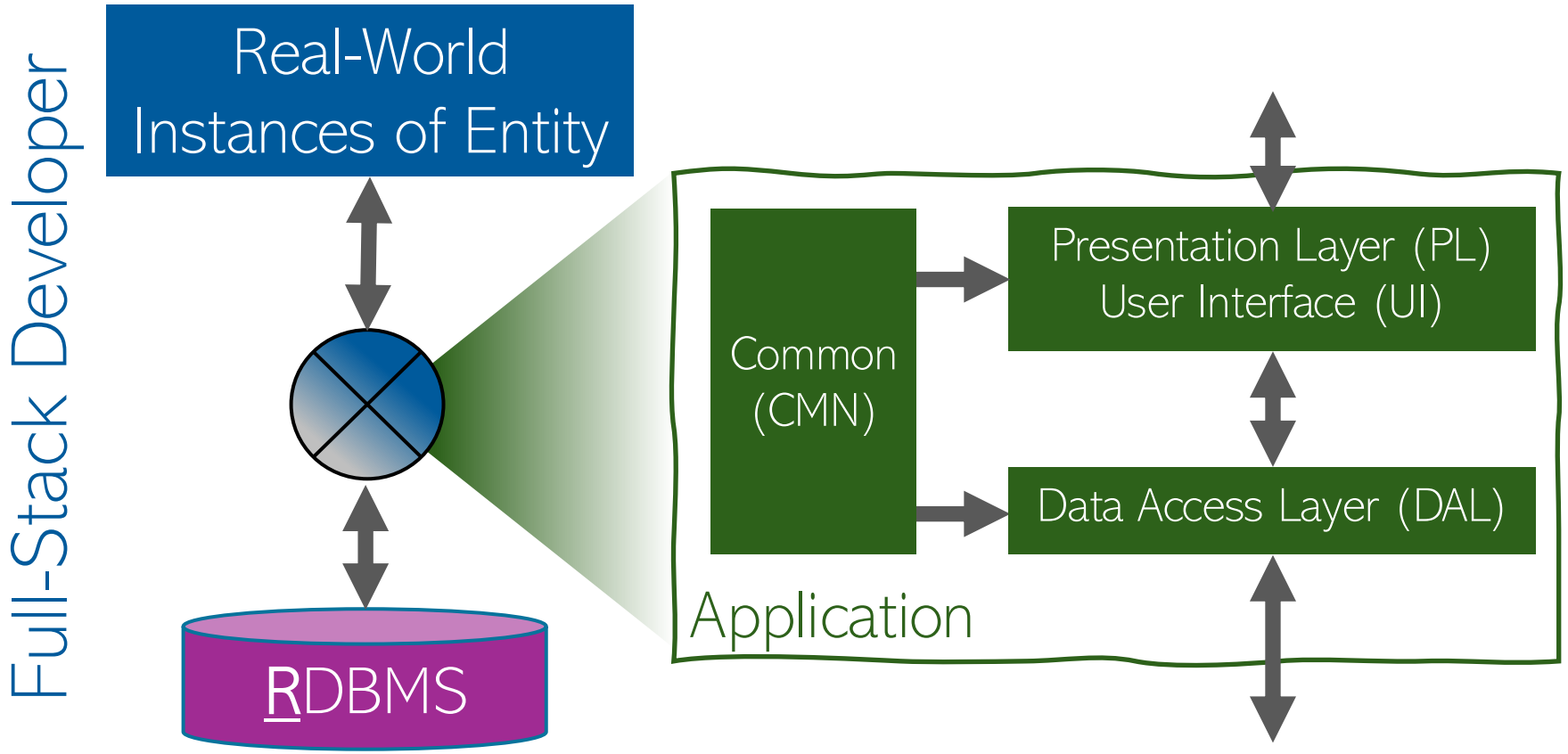


→ Data Access Layer (DAL)

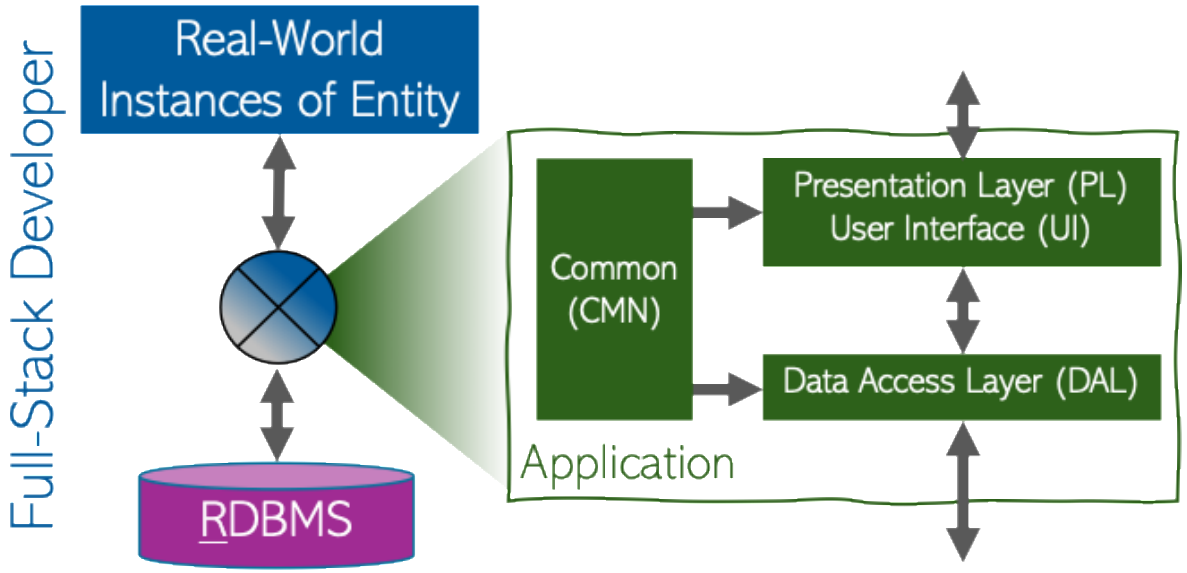


The Matrix (1999), He is the One.

SQL: DML via DAL

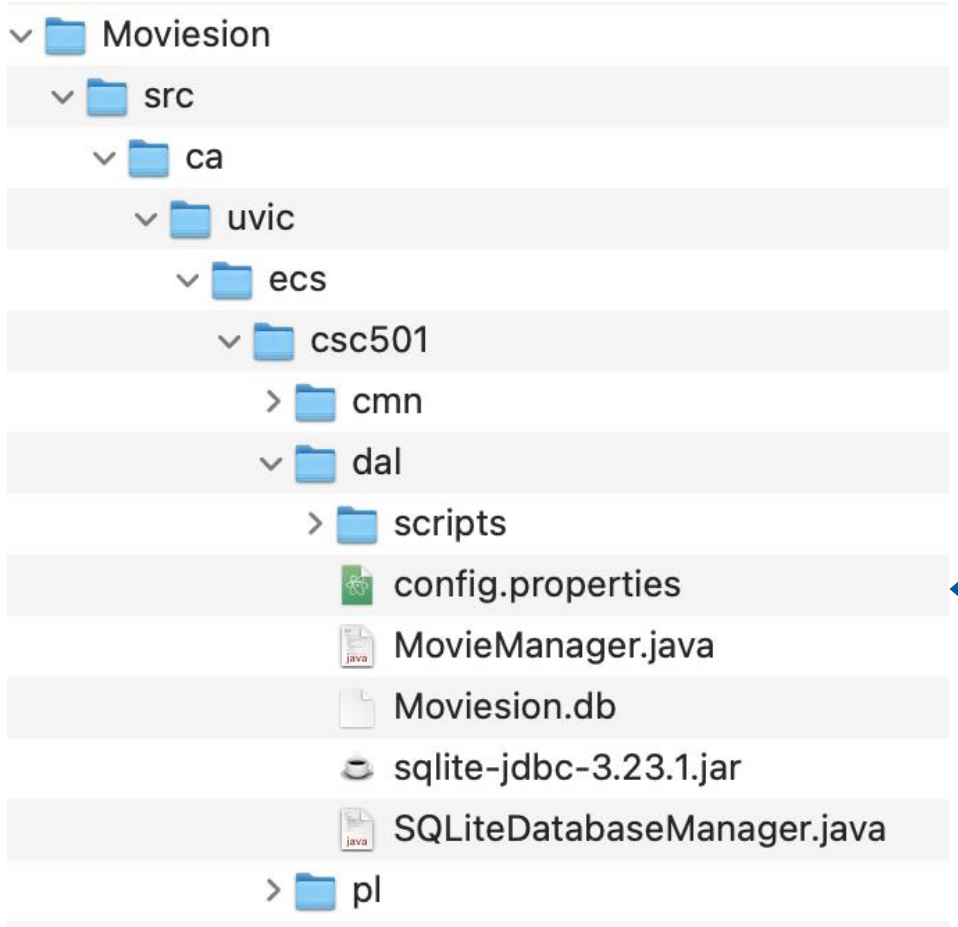


SQL: DML via DAL



- Moviesion
  - src
    - ca
      - uvic
        - ecs
          - csc501
            - cmn
              - Director.java
              - Movie.java
            - dal
              - scripts
                - adhoc
                - schema.sql
              - config.properties
              - MovieManager.java
              - Moviesion.db
              - sqlite-jdbc-3.23.1.jar
              - SQLiteDatabaseManager.java
            - pl
              - Moviesion.java

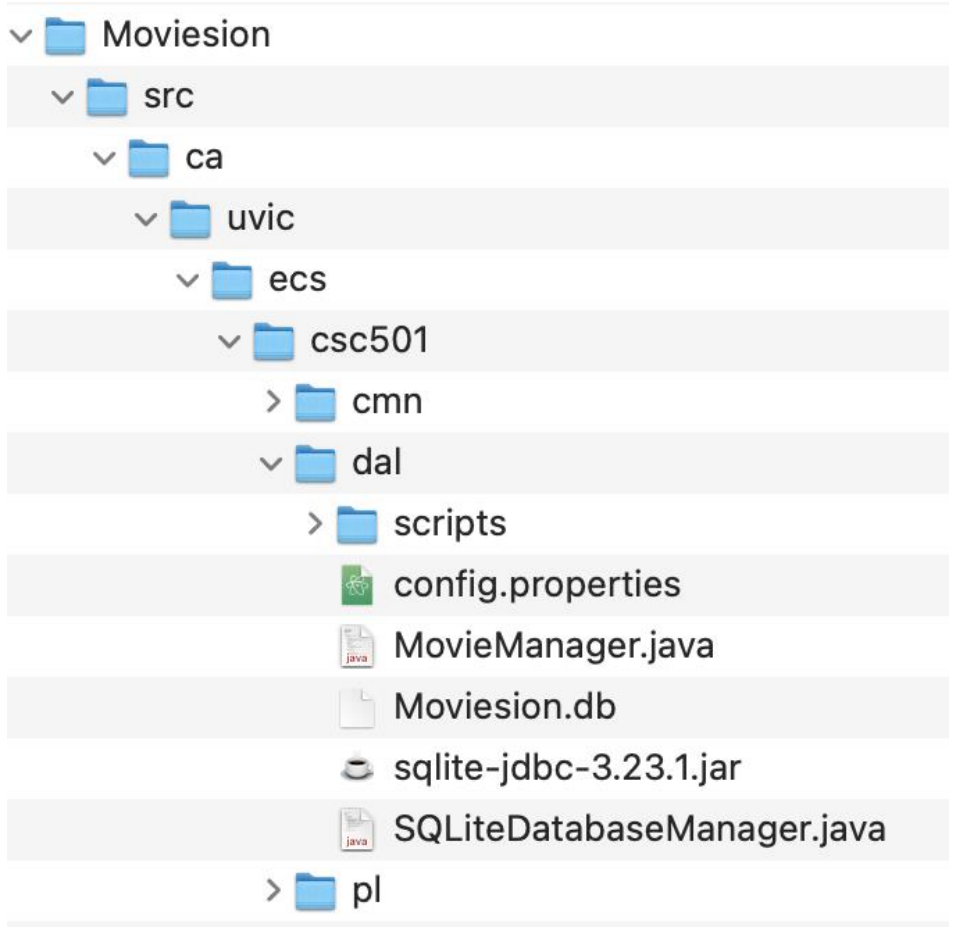
26 SQL: DML via DAL



1

Settings to connect to RDBMS:
Location, Username, Password, DatabaseName

27 SQL: DML via DAL

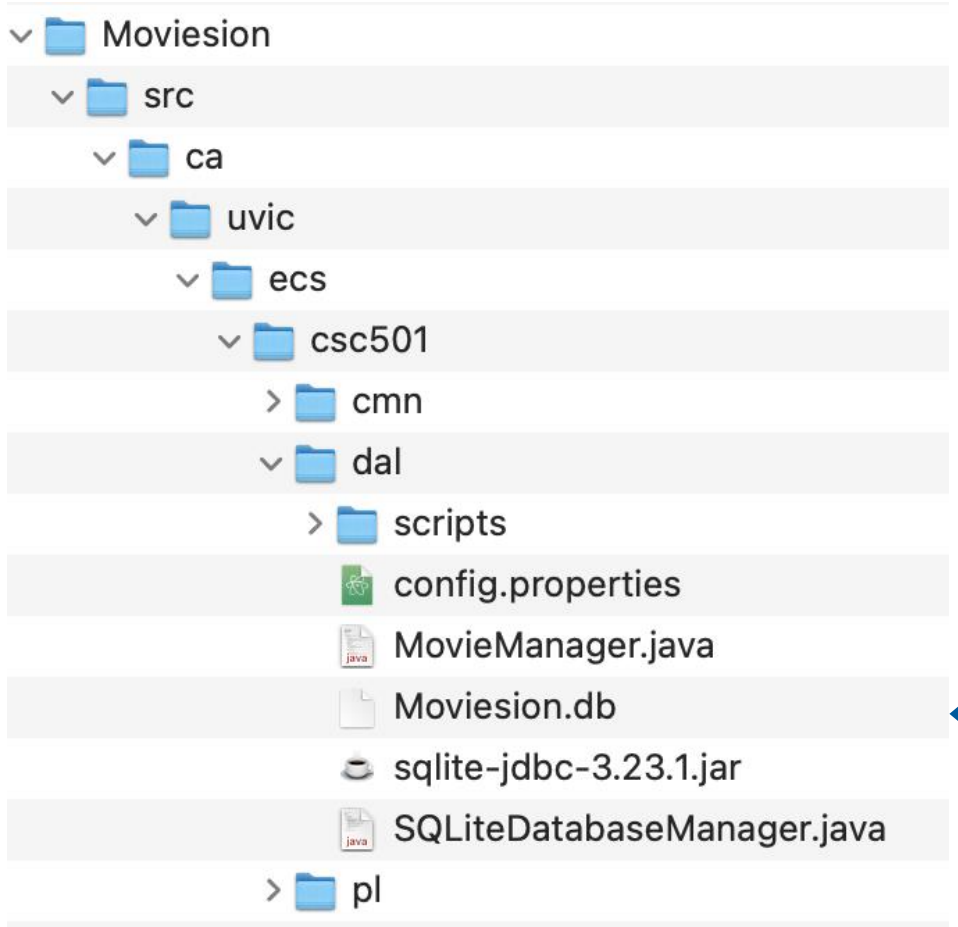


2

Anything related to DBMS connection:
Loading SQLite library
Loading settings from config file
Create a connection

28

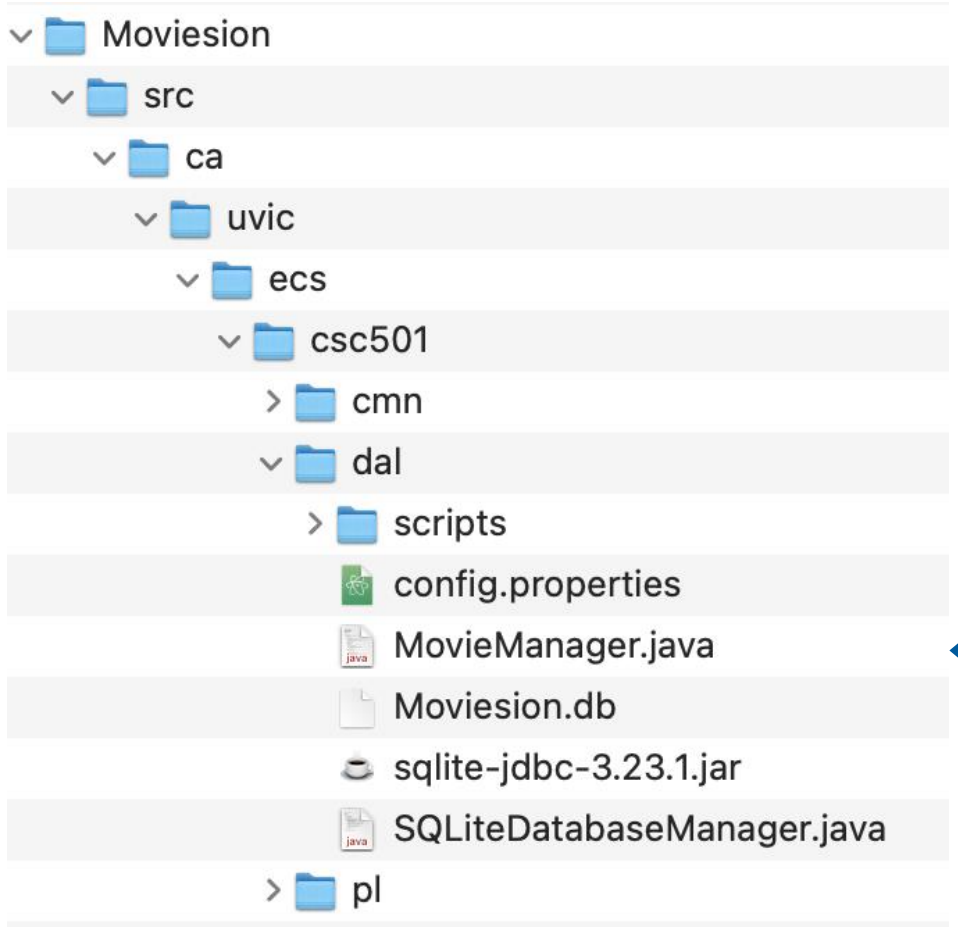
SQL: DML via DAL



3

The database, including all relations.

SQL: DML via DAL

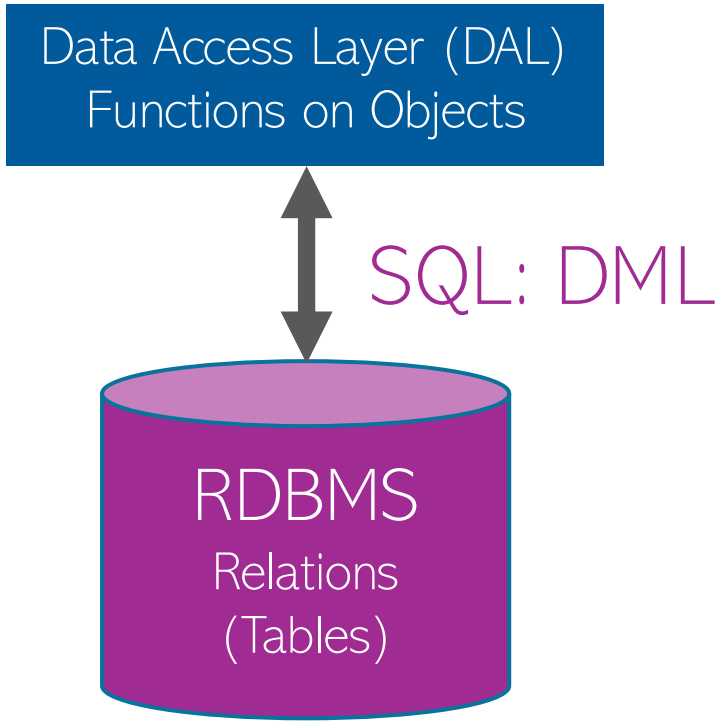


4

Anything related to Movie relation (table):

addMovie()	INSERT
editMovie()	UPDATE
removeMovie()	DELETE
getMovie()	SELECT

SQL: DML via DAL by Object Relational Mapping (ORM)



Object

Relation (Table)

```
public void add(Movie newMovie){
    ...
    String sql = "INSERT INTO Director(Name) VALUES(?)";
    ps = db.prepareStatement(...);
    ps.setString(1, newMovie.getName());
    rowsAffected = ps.executeUpdate();
    ...
}
```

SQL: DML

Data Manipulation Language to

```
INSERT
UPDATE
DELETE
SELECT
```

to/from relations (tables).

SQL: DML: INSERT

```
INSERT INTO TableName(c1, c2, ....) VALUES (v1, v2, ...);
```

- The number of columns and values must be same.
- If column list omitted, the order in original table is assumed.
- Data types of values must be compatible with the respective columns.

```
INSERT INTO Director(FirstName, LastName) VALUES ('Alfred', 'Hitchcock');
INSERT INTO Director VALUES ('Alfred', 'Hitchcock');
INSERT INTO Director(FirstName, LastName) VALUES (2, 'Hitchcock');
```

SQL: DML: UPDATE

```
UPDATE TableName SET c1=v1, c2=v2, ... [WHERE θ];
```

- Data types of values must be with the respective columns.
- Only rows which satisfy the θ condition will be updated.
- If there is no θ, all rows will be updated!

```
UPDATE Director SET LastName='Hitchkok' WHERE Id = 1;
1 rows affected
UPDATE Director SET LastName='Hitchkok' WHERE LastName = 'Hitchcock';
3 rows affected
UPDATE Director SET LastName='Hitchkok'
36 rows affected
```

SQL: DML: UPDATE

UPDATE TableName SET c1=v1, c2=v2, ... [WHERE θ];

Project Policy: Codes about UPDATE without WHERE will not be merged.

SQL: DML: DELETE

DELETE FROM TableName [WHERE θ];

- Only rows which satisfy the θ condition will be deleted.
- If there is no θ, all rows will be deleted.

```
DELETE FROM Director WHERE Id = 1;
1 rows affected
DELETE FROM Director WHERE LastName = 'Hitchcock';
3 rows affected
DELETE FROM Director
36 rows affected
```

41

SQL: DML: DELETE

DELETE FROM TableName [WHERE θ];

Project Policy: Codes about DELETE without WHERE will not be merged.
Project Policy: Codes about DELETE without WHERE will not be merged.
Project Policy: Codes about DELETE without WHERE will not be merged.
Project Policy: Codes about DELETE without WHERE will not be merged.

SQL: DML: Data Integrity

DML manipulated tuples in a database. So, after each operation, DBMS checks the integrity of database:

I. Domain Integrity
II. Entity Integrity
III. Referential Integrity
IIII. User-defined Integrity

DML will fail if they violate (conflict with) any integrity constraints!

SQL: DML: Data Integrity

DML manipulated tuples in a database. So, after each operation, DBMS checks the integrity of database:

I. Domain Integrity (Data Type Integrity)
II. Entity Integrity
III. Referential Integrity
IIII. User-defined Integrity

DML will fail if they violate (conflict with) any integrity constraints!

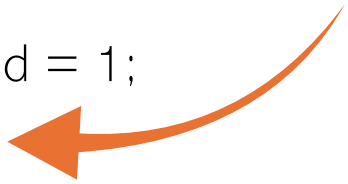
SQL: DML: Data Integrity: Domain Integrity

- All columns must have a defined domain (data type).
- Values in a column must comply with domain (data type).
- This includes `NULL` or `NOT NULL`.

```
UPDATE Director SET LastName=12.25 WHERE Id = 1;
```
data type mismatch!

```
INSERT INTO Director(FirstName, LastName) VALUES (1, NULL, 'Hitchcock');
```
`NOT NULL` constraint failed: Director.FirstName

```
UPDATE Director SET LastName=12.25 WHERE Id = 1;
```
SQLite > 1 rows affected.



SQL: DML: Data Integrity

DML manipulated tuples in a database. So, after each operation, DBMS checks the integrity of database:

I. Domain Integrity (Data Type Integrity)
II. Entity Integrity (Primary Key Integrity)
III. Referential Integrity
IIII. User-defined Integrity

DML will fail if they violate (conflict with) any integrity constraints!

SQL: DML: Data Integrity: Entity Integrity

- Every relation (table) must have a Primary Key.
- The column(s) of Primary Key must be `UNIQUE` and `NOT NULL`.

```
INSERT INTO Director(Id, FirstName, LastName) VALUES (1,'Alfred', 'Hitchcock');
INSERT INTO Director(Id, FirstName, LastName) VALUES (1,'Alfred', 'Hitchcock');
UNIQUE constraint failed: Director.Id
```

For our surrogate key policy during entity-to-relational mapping, we never face this problem → all relations (tables) has Id as PK, autogenerated, `NOT NULL`.

SQL: DML: Data Integrity

DML manipulated tuples in a database. So, after each operation, DBMS checks the integrity of database:

I. Domain Integrity (Data Type Integrity)
II. Entity Integrity (Primary Key Integrity)
III. Referential Integrity (Foreign Key Integrity)
IIII. User-defined Integrity

DML will fail if they violate (conflict with) any integrity constraints!

SQL: DML: Data Integrity: Referential Integrity

- Foreign Key values can only be:
  - Primary Key values of another table;
  - NULL.

```
INSERT INTO Director(Id, FirstName, LastName) VALUES (1,'Alfred', 'Hitchcock');
INSERT INTO Movie(Id, Title) VALUES (1, 'The Birds');
INSERT INTO MovieDirector(MovieId, DirectorId) VALUES (1, 1);

INSERT INTO MovieDirector(MovieId, DirectorId) VALUES (10, 1);
FOREIGN KEY constraint failed, FK_MovieDirector_MovieId_2_Movie_Id

DELETE FROM Director;
FOREIGN KEY constraint failed
```



Why?

SQL: DML: Data Integrity

DML manipulated tuples in a database. So, after each operation, DBMS checks the integrity of database:

I. Domain Integrity (Data Type Integrity)
II. Entity Integrity (Primary Key Integrity)
III. Referential Integrity (Foreign Key Integrity)
IIII. User-defined Integrity

DML will fail if they violate (conflict with) any integrity constraints!

SQL: DML: Data Integrity: User-defined Integrity

Any other constraints specified by a database designer such as Candidate Keys (`UNIQUE`), `CHECK`, ...

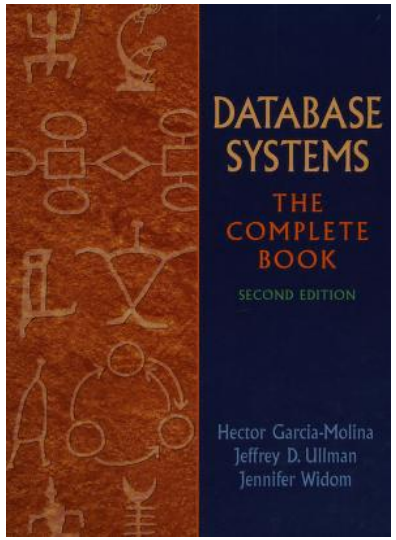
```
INSERT INTO Movie(Title, ReleaseDate) VALUES ('The Birds', 1963);
INSERT INTO Movie(Title, ReleaseDate) VALUES ('The Birds', 1964);
UNIQUE constraint failed: Movie.Title
```

SQL: DML: Data Integrity: User-defined Integrity

- Any constraints that are missing in ER and Relational modeling, can be defined using `CHECK`.

- However, few designer does that. Why?
- Few designers even disable data integrity. Why?

Outline: W07: Physical Modeling: SQL



CH06 (2nd Ed.)

I) Introduction

II) Data Definition Language (DDL)

III) Data Modification Language (DML)

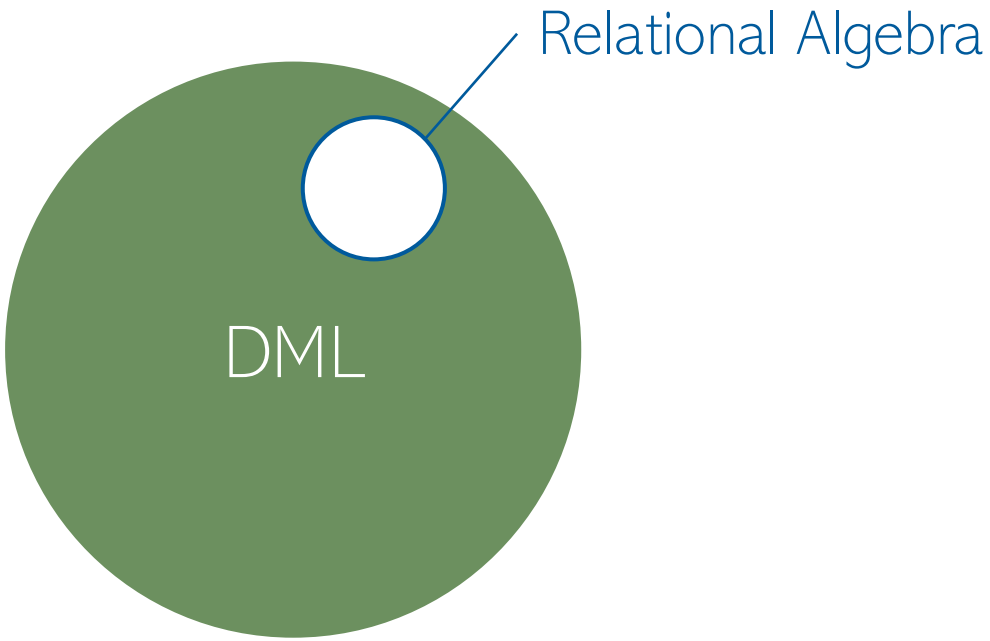
IIII) Data Query Language (`SELECT`)

Hands-On

Outline: W07: Physical Modeling: SQL

Relational algebra , i.e.,

- Project (π)
- Select (σ)
- Cartesian Product ($\times$)
- Union ($\cup$)
- Intersection ($\cap$)
- Set Difference (\), Division, Joins, …

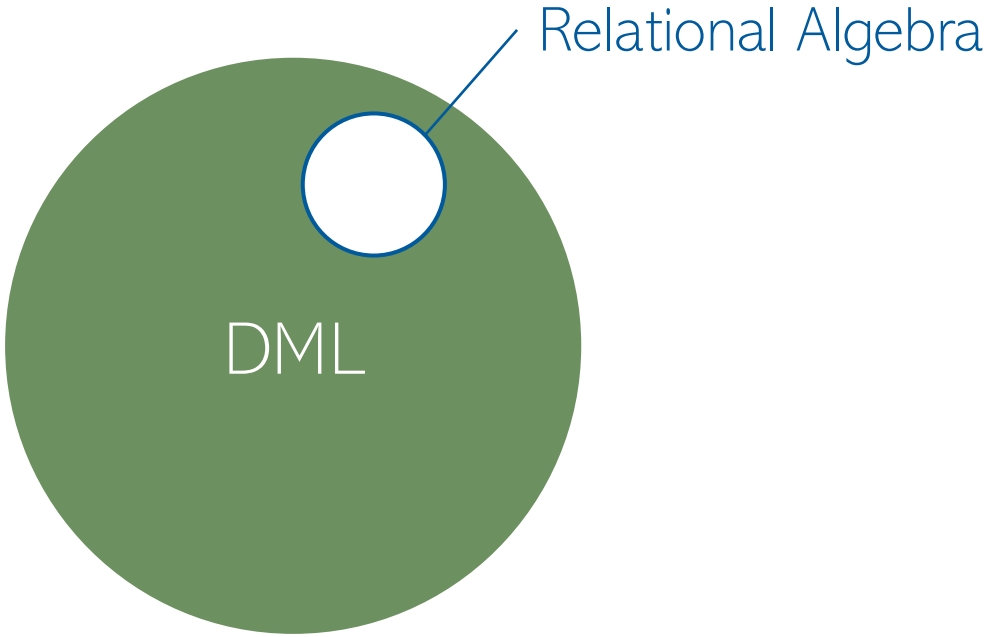


Only by one statement (command): `SELECT`

Outline: W07: Physical Modeling: SQL

Relational algebra , i.e.,

5 `SELECT DISTINCT` Columns
1 `FROM` Tables
2 [`WHERE` θ]
3 [`GROUP BY` Columns]
4 [`HAVING` θ′]
6 [`ORDER BY` Columns [`ASC` | `DESC`]]
7 [`LIMIT` # [`OFFSET` #]]



SQL: Projection (π)

What are directors' names and their birth places?

$\pi_{\text{FirstName, LastName, MovieCount}}(\text{Director})$

```
5 SELECT FirstName, LastName, MovieCount
1 FROM Director
```

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803

SQL: Projection (π) by Aggregation Functions

COUNT, SUM, MIN, MAX, AVG

How many directors, and the birth date of the oldest director?

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803

5 `SELECT COUNT(Id) AS Count, MIN(BirthDate) AS Oldest`
1 `FROM Director`

Count	Oldest
3	Aug. 13, 1899

SQL: Projection (π) by Aggregation Functions

`COUNT`, `SUM`, `MIN`, `MAX`, `AVG`

Who is the oldest director?

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803

5 `SELECT MIN`(BirthDate) `AS` Oldest, Id `AS` OldestDirectorId ✗

1 `FROM` Director

60 SQL: Selection (σ)

Who are American directors?

$$\sigma_{\text{BirthPlace}=\text{'USA'}}(\text{Director})$$

```
3 SELECT Id, FirstName, LastName, BirthDate, BirthPlace, BestMovieId
1 FROM Director
2 WHERE BirthPlace= 'USA'
```

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803

61

SQL: Selection (∫) by Aggregation Functions

COUNT, SUM, MIN, MAX, AVG

How many American directors?

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803

```
3 SELECT COUNT(Id) AS AmericanDirectorsCount
1 FROM Director
2 WHERE BirthPlace= 'USA'
```

AmericanDirectorsCount
2

62 SQL: Selection ($\int$) + `GROUP BY` + Aggregation

5 `SELECT DISTINCT` Columns
1 `FROM` Tables
2 [`WHERE` θ]
3 [`GROUP BY` Columns]
4 [`HAVING` θ']

SQL: Selection (∫) then GROUP BY then Aggregation

How many directors per countries in 20th century?

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803
4	Roman	Polanski	Aug. 18, 1933	France	871

```
5 SELECT BirthPlace AS Country, COUNT(Id) AS DirectorsCount
1 FROM Director
2 WHERE BirthDate>=1900
3 GROUP BY BirthPlace
```

Country	DirectorsCount
USA	2
France	1

SQL: Selection (∫) then GROUP BY then Aggregation

What are countries with at least two directors in 20th century?

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803
4	Roman	Polanski	Aug. 18, 1933	France	871

```
5 SELECT BirthPlace AS Country, COUNT(Id) AS DirectorsCount
1 FROM Director
2 WHERE BirthDate>=1900
3 GROUP BY BirthPlace
4 HAVING COUNT(Id) >= 2
```

Country	DirectorsCount
USA	2

65 SQL: Sorting

5 `SELECT DISTINCT` Columns
1 `FROM` Tables
2 [`WHERE` θ]
3 [`GROUP BY` Columns]
4 [`HAVING` θ']
6 [`ORDER BY` Columns [`ASC` | `DESC`]]

SQL: Sorting

How many directors per countries in 20th century?

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803
4	Roman	Polanski	Aug. 18, 1933	France	871

```
5 SELECT BirthPlace AS Country, COUNT(Id) AS DirectorsCount
1 FROM Director
2 WHERE BirthDate>=1900
3 GROUP BY BirthPlace
6 ORDER BY DirectorsCount DESC
```

Country	DirectorsCount
USA	2
France	1

67 SQL: Sorting

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803
4	Roman	Polanski	Aug. 18, 1933	France	871

5 1 6

```
SELECT Id, FirstName, LastName, BirthDate, BirthPlace
FROM Director
ORDER BY BirthPlace, BirthDate DESC
```

Director				
Id	FirstName	LastName	BirthDate	BirthPlace
2	Alfred	Hitchcock	Aug. 13, 1899	England
4	Roman	Polanski	Aug. 18, 1933	France
3	Clint	Eastwood	May 31, 1930	USA
1	Stanley	Kubrick	Jul. 26, 1928	USA

SQL: Paging: Top-N tuples after skipping top-M ones

5 `SELECT DISTINCT` Columns
1 `FROM` Tables
2 [`WHERE` θ]
3 [`GROUP BY` Columns]
4 [`HAVING` θ′]
6 [`ORDER BY` Columns [`ASC` | `DESC`]]
7 [`LIMIT` N [`OFFSET` M]]

SQL: Paging

Top-N tuples After skipping Top-M ones

5 `SELECT` FirstName, LastName
1 `FROM` Director
6 `ORDER BY` LastName
7 `LIMIT` 10 `OFFSET` 2×10

FirstName	LastName
Pedro	Almodóvar
Wes	Anderson
Darren	Aronofsky
Paul Thomas	Anderson
Robert	Altman
Ingmar	Bergman
Kathryn	Bigelow
James	Cameron
Sofia	Coppola
Francis Ford	Coppola
Joel	Coen
Ethan	Coen
Alfonso	Cuarón
Guillermo	del Toro
Lana	Wachowski
Lilly	Wachowski
Claire	Denis
Céline	Sciamma
David	Fincher
John	Ford
Jean-Luc	Godard
Greta	Gerwig
Werner	Herzog
Alfred	Hitchcock
Peter	Jackson
Bong	Joon-ho
Alejandro	González Iñárritu
Stanley	Kubrick
Akira	Kurosawa
Ang	Lee
Spike	Lee
Richard	Linklater
George	Lucas
Terrence	Malick
Christopher	Nolan
Jordan	Peele
Roman	Polanski
Satyajit	Ray
Martin	Scorsese
Steven	Spielberg
Quentin	Tarantino
Andrei	Tarkovsky
Denis	Villeneuve
Lars	von Trier
Orson	Welles
Chloé	Zhao

Skipping Top-M=2×N
2 pages of N tuples

Top-N tuples
1 page of N tuples
3rd page

SQL: Paging

Who is the oldest director?

Director					
Id	FirstName	LastName	BirthDate	BirthPlace	BestMovieId
1	Stanley	Kubrick	Jul. 26, 1928	USA	1
2	Alfred	Hitchcock	Aug. 13, 1899	England	203
3	Clint	Eastwood	May 31, 1930	USA	803

```
5 SELECT FirstName, LastName, BirthDate, BirthPlace
1 FROM Director
6 ORDER BY BirthDate
7 LIMIT 1
```



FirstName	LastName	BirthDate	BirthPlace
Alfred	Hitchcock	Aug. 13, 1899	England
Stanley	Kubrick	Jul. 26, 1928	USA
Clint	Eastwood	May 31, 1930	USA

SQL: Cartesian Product (×)

5 `SELECT DISTINCT` Columns
1 `FROM` Tables → Table1, Table2, ..., TableN
2 [`WHERE` θ]
3 [`GROUP BY` Columns]
4 [`HAVING` θ′]
6 [`ORDER BY` Columns [`ASC` | `DESC`]]
7 [`LIMIT` N [`OFFSET` M]]